Microservices with Docker *and* Kubernetes: An Overview

Docker is an open source platform that's used to build, ship and run distributed services. Kubernetes is an open source orchestration platform for automating deployment, scaling and the operations of application containers across clusters of hosts. Microservices structure an application into several modular services. Here's a quick look at why these are so useful today.

The Linux operating system has become very stable now and is capable of cleanly sandboxing processes, to execute processes easily; it also comes with better name space control. This has led to the development and enhancement of various container technologies. Some of the features/characteristics of the Linux OS that help container development are:

- Only the required libraries get installed in their respective containers.
- Custom containers can be built easily.
- During the initial days, LXC (Linux container) was very popular and was the foundation stone for the development of various other containers.
- Name space and control groups (process level isolation).

A brief history of containers is outlined in Table 1.

Some of the advantages of containers are:

- Containers are more lightweight compared to virtual machines (VMs).
- The container platform is used in a concise way to build Docker (which is one of the container standards; it is actually a static library and is a daemon running inside the Linux OS).
- Containers make our applications portable.
- Containers can be easily shipped, built and deployed.

Containers

Containers are an encapsulation of an application with its dependencies. They look like lightweight VMs but that is not the case. A container holds an isolated instance of an operating system, which is used to run various other applications.

The architecture diagram of Docker-container in Figure 1 shows how each of the individual components are interconnected.

Various components of the Docker-container architecture

The Docker daemon (generally referred to as *dockerd*) listens for Docker API requests and manages Docker objects such as images, containers, networks and volumes. A daemon